

ACTIVIDAD 10 DE 46 - GESTION DE SECRETOS: GITLEAKS Y CREDENCIALES FUERA DEL CODIGO

Segunda hora de clase (50 minutos) | Trabajo practico asistido con IA

Herramienta principal: opencode (CLI). Alternativas: ChatGPT, Claude, Copilot, Gemini.

Base normativa / marco de referencia: Gestion de secretos y credenciales (dominio GA/GH del Marco Agesic); buenas practicas OWASP

1. Contexto

Un secreto en el codigo (token, clave, password) es una fuga en potencia. Hoy moveras las credenciales de tu API a un archivo .env, configuraras .gitignore, y usaras gitleaks para buscar secretos filtrados en el repositorio. Simularas una fuga, la detectarás y la remediaras (rotacion).

2. Marco teorico

¿Que es un secreto?

Es informacion sensible que autentica o autoriza: contraseñas, tokens, claves API, certificados privados.

¿Por que es peligroso en el codigo?

Si el codigo se sube a un repositorio (publico o interno), el secreto queda expuesto a quien tenga acceso. Es una fuga en potencia.

Buenas practicas

- Usar variables de entorno o gestores de secretos (nunca en el codigo).
- No versionar archivos .env (agregarlos a .gitignore).
- Rotar (cambiar) los secretos periodicamente y ante cualquier sospecha.
- Gitleaks y herramientas similares buscan secretos accidentalmente versionados.

Si un secreto se expuso

Se asume comprometido: se rota inmediatamente, se revisa si fue usado y se notifica segun el procedimiento.

3. Objetivos de aprendizaje

- Mover secretos a variables de entorno (.env).
- Configurar .gitignore para excluir secretos.

- Instalar gitleaks y escanear el repositorio.
- Detectar, remediar y rotar un secreto filtrado.

4. Herramientas del día (instalacion paso a paso)

gitleaks (escaneo gratuito de secretos)

Busca tokens, claves y passwords en repositorios Git. Detecta secretos historicos (en cualquier commit).

1. Instala: ``winget install gitleaks.gitleaks``.
2. Verifica: ``gitleaks version``.

python-dotenv

Carga las variables de entorno desde un archivo `.env` en Python.

3. Instala: ``pip install python-dotenv``.
4. Uso: ``from dotenv import load_dotenv`` y ``load_dotenv()``.

5. Desarrollo paso a paso

Paso 1: Crear `.env`

Crea ``codigo/.env`` con tu token de la API: `TOKEN_API=mi_token_secreto_123`.

Paso 2: Cargar en codigo

Modifica `app.py` para leer el token desde el entorno con `os.getenv`.

Paso 3: Excluir `.env`

Agrega ``*.env`` al `.gitignore` y haz commit de esta correccion.

Paso 4: Instalar y escanear

Ejecuta ``gitleaks detect`` en la raiz del repositorio.

Paso 5: Simular una fuga

Crea intencionalmente un archivo con un secreto falso, haz commit, y vuelve a escanear.

Paso 6: Detectar la fuga

Observa como gitleaks reporta el archivo y el commit.

Paso 7: Remediar y rotar

Elimina el archivo, rota el secreto (cambialo por uno nuevo) y re-escanee hasta limpiar.

Paso 8: Documentar y reflexionar

Escribe el informe y tu reflexion.

6. Prompts sugeridos para la IA (opencode)

Copia y adapta estos mensajes en opencode (comando interactivo o `opencode` en terminal).

Dame una guia de buenas practicas de gestion de secretos para un equipo de desarrollo: donde guardarlos, como rotarlos y como escanear el repositorio.

Explicame el informe de gitleaks: [pega la salida]. ¿Que tipo de secreto es y por que es peligroso?

¿Que pasos debe seguir un equipo si descubre que una clave se filtro a un repositorio publico? Incluye rotacion y comunicacion.

7. Documentacion del RSI (bitacora)

Registra en `bitacora.md` de la carpeta de la actividad:

- Secretos movidos al .env.
- Resultado del escaneo con gitleaks (antes y despues).
- Procedimiento de rotacion aplicado.
- Regla del equipo: los secretos nunca van al repositorio.

8. Evidencias a capturar y guardar

Guarda cada evidencia en la carpeta `evidencias/` con el nombre EVI-YYYY-MM-DD-NN-descripcion.ext y registrala en la bitacora.

- EVI-...-01-gitleaks-limpio.txt (escaneo inicial).
- EVI-...-02-gitleaks-fuga.txt (escaneo con la fuga).
- EVI-...-03-gitleaks-rotado.txt (escaneo final limpio).
- EVI-...-04-gitignore.png (.env excluido).

9. Informe segun el marco de referencia

Elabora el informe de la actividad (archivo `informe.md`) que incluya:

- Procedimiento de gestion de secretos aplicado.
- Fuga simulada: como se detecto y donde.
- Remediacion y rotacion realizada.
- Politica propuesta de secretos.

10. Entregable de la clase

API sin secretos en código + .env + escaneos de gitleaks antes/después + procedimiento de rotación.

Anexo A - Informe completo para presentar al BCU

Plantilla del informe ejecutivo que el RSI entregaría al Banco Central del Uruguay (BCU) por la actividad 10 - Gestión de secretos: gitleaks y credenciales fuera del código. Completa los campos [entre corchetes] con los resultados reales de la actividad y adjunta las evidencias EVI-...

Encabezado del informe

Institución: Banco Ficticio del Uruguay

Responsable de Seguridad de la Información (RSI): [Nombre y cargo]

Destinatario: Banco Central del Uruguay - Superintendencia de Instituciones Financieras

Asunto: Actividad 10 - Gestión de secretos: gitleaks y credenciales fuera del código

Fecha: [dd/mm/aaaa] | Versión: 1.0 | Clasificación: Confidencial

1. Resumen ejecutivo

[2 a 5 párrafos breves: que se realizó, en qué marco normativo, que se encontró, que se corrigió y el estado actual. Debe poder leerse en 1 minuto por un directivo.]

2. Marco normativo y regulatorio aplicable

- Norma de Seguridad de la Información y Ciberseguridad del BCU (SF.SEG.08).
- Recopilación de Normas de Regulación y Control del Sistema Financiero (RNRCFS).
- Circular sobre gestión de incidentes y notificación de eventos de ciberseguridad.
- Normativa interna del Banco Ficticio del Uruguay: [política de seguridad, plan de trabajo RSI].

3. Alcance y metodología

- Alcance: [activos, procesos, sistemas o áreas cubiertos por la actividad].
- Método: [pruebas técnicas, revisión documental, simulación, escaneo, etc.].
- Herramientas utilizadas: gitleaks (escaneo gratuito de secretos), python-dotenv.
- Período de ejecución: [fecha de inicio] a [fecha de fin].

4. Descripción de la actividad

Un secreto en el código (token, clave, password) es una fuga en potencia. Hoy moverás las credenciales de tu API a un archivo .env, configurarás .gitignore, y usarás gitleaks para buscar secretos filtrados en el repositorio. Simularás una fuga, la detectarás y la remediarás (rotación).

5. Resultados obtenidos

- Objetivo: Mover secretos a variables de entorno (.env). -> [Logrado / Parcial / No logrado]
- Objetivo: Configurar .gitignore para excluir secretos. -> [Logrado / Parcial / No logrado]
- Objetivo: Instalar gitleaks y escanear el repositorio. -> [Logrado / Parcial / No logrado]
- Objetivo: Detectar, remediar y rotar un secreto filtrado. -> [Logrado / Parcial / No logrado]

6. Riesgos identificados y evaluacion

Tabla de riesgos detectados con impacto en la institucion financiera:

ID Riesgo	Descripcion	Probabilidad	Impacto	Nivel	Tratamiento
R-01	[riesgo encontrado]	[A/M/B]	[A/M/B]	[C/E/M]	[Mitigado / Aceptado / En plan]
R-02	[riesgo encontrado]	[A/M/B]	[A/M/B]	[C/E/M]	[Mitigado / Aceptado / En plan]

7. Controles implementados

Descripcion de los controles aplicados y su estado:

- Control: Mueve secretos a variables de entorno -> [Implementado / Parcial / Pendiente]
- Control: Configura .gitignore correctamente -> [Implementado / Parcial / Pendiente]
- Control: Instala y ejecuta gitleaks -> [Implementado / Parcial / Pendiente]
- Control: Detecta, remedia y rota un secreto -> [Implementado / Parcial / Pendiente]
- Control: Documenta la politica de secretos -> [Implementado / Parcial / Pendiente]

8. Incidentes de seguridad relacionados

[Ninguno en esta actividad. O bien: fecha, tipo, clasificacion, datos afectados, impacto y medidas adoptadas. Si corresponde, nota que ya se notifico al BCU en los plazos reglamentarios.]

9. Indicadores y metricas de la actividad

- Tiempo de ejecucion: [X] horas.
- Hallazgos: [X] altos, [X] medios, [X] bajos.
- Hallazgos corregidos: [X] / [X].
- Cobertura: [X]% de los activos del alcance.

10. Plan de accion y proximos pasos

Accion	Responsable	Plazo	Estado
[accion pendiente]	[cargo]	[dd/mm/aaaa]	Pendiente

11. Conclusion

[Conclusion ejecutiva: postura de seguridad de la institucion tras la actividad y compromiso de seguimiento.]

12. Firma y aprobacion

Firma del RSI: _____ Fecha: [dd/mm/aaaa]

Visto bueno del area de negocio / Direccion: _____

Anexos: evidencias EVI-... registradas en la bitacora.

Anexo B - Como presentar la actividad a AGESIC

Guia de estructura y contenido para comunicar la actividad 10 - Gestion de secretos: gitleaks y credenciales fuera del codigo a la Agencia para el Desarrollo del Gobierno de Gestion Electronica y la Sociedad de la Informacion (AGESIC), alineada al marco de ciberseguridad de referencia para Uruguay.

Encuadre institucional

- Organismo: Banco Ficticio del Uruguay, dependencia publica.
- RSI: [Nombre y cargo] | Correo: [correo institucional].
- Fecha de presentacion: [dd/mm/aaaa] | Periodo cubierto: [periodo].

1. Presentacion del responsable

Quien es el RSI, su rol, a que direccion reporta y como se vincula con la gestion de la seguridad de la informacion en el Banco Ficticio del Uruguay.

2. Objetivo de la actividad

Un secreto en el codigo (token, clave, password) es una fuga en potencia. Hoy moveras las credenciales de tu API a un archivo .env, configuraras .gitignore, y usaras gitleaks para buscar secretos filtrados en el repositorio. Simularas una fuga, la detectaras y la remediaras (rotacion).

- Objetivo: Mover secretos a variables de entorno (.env).
- Objetivo: Configurar .gitignore para excluir secretos.
- Objetivo: Instalar gitleaks y escanear el repositorio.
- Objetivo: Detectar, remediar y rotar un secreto filtrado.

3. Marco de referencia utilizado

- Marco normativo de ciberseguridad aplicable (AGESIC / normativa nacional).
- Ley 18.331 de Proteccion de Datos Personales y su Decreto Reglamentario.
- Guia o documento de base: Gestion de secretos y credenciales (dominio GA/GH del Marco Agesic); buenas practicas OWASP
- Estandares de buenas practicas: [ISO/IEC 27001, NIST CSF, OWASP, etc.].

4. Metodologia y herramientas

Como se desarrollo la actividad y con que herramientas (gitleaks (escaneo gratuito de secretos), python-dotenv).

5. Resultados y evidencias

Descripcion de lo obtenido, con referencia a las evidencias guardadas:

- Evidencia: EVI-...-01-gitleaks-limpio.txt (escaneo inicial).
- Evidencia: EVI-...-02-gitleaks-fuga.txt (escaneo con la fuga).
- Evidencia: EVI-...-03-gitleaks-rotado.txt (escaneo final limpio).
- Evidencia: EVI-...-04-gitignore.png (.env excluido).

6. Hallazgos, riesgos y recomendaciones

Presentacion de hallazgos ordenados por criticidad y las recomendaciones asociadas:

ID	Hallazgo	Criticidad	Recomendacion	Estado
H-01	[hallazgo]	[C/A/M/B]	[recomendacion]	[Abierto / Cerrado]

7. Gestion de incidentes (si aplica)

- Tipificacion del evento y datos involucrados.
- Notificacion a la Unidad Reguladora y de Control de Datos Personales si se vieron afectados datos personales.
- Lecciones aprendidas y mejoras al plan de respuesta.

8. Plan de mejora y continuidad

Mejora planificada	Responsable	Plazo	Prioridad
[mejora]	[cargo]	[dd/mm/aaaa]	[A/M/B]

9. Compromisos y firmas

El Banco Ficticio del Uruguay se compromete a [seguimiento, capacitacion, revision periodica] y presenta este informe para conocimiento de AGESIC.

Firma del RSI: _____ Sello del organismo: _____

11. Rubrica de evaluacion

Criterio	Logrado (3)	En desarrollo (2)	No logrado (1)
Mueve secretos a variables de entorno			
Configura .gitignore correctamente			
Instala y ejecuta gitleaks			
Detecta, remedia y rota un secreto			
Documenta la politica de secretos			

12. Checklist de cierre

- [] Los secretos estan solo en .env.
- [] Ejecuté gitleaks detect.

- [] Remedie y roté la fuga simulada.

13. Reflexion sobre el uso de IA

¿Por que un secreto historico en Git sigue siendo peligroso aunque lo borres? ¿Que haria un atacante al encontrar un token filtrado?